

DAA – TOPIC 6

Experiment 1: Merge Sort with Comparison Count

• **Aim:** To implement Merge Sort, count element comparisons during merging, and display the sorted array with the total comparison count.

• **Algorithm:**

1. Divide the array into two halves recursively.
2. Sort each half using Merge Sort.
3. Merge the sorted halves.
4. Increment the comparison counter whenever two elements are compared.
5. Display the sorted array and total comparisons.

PROGRAM

Python Program

```
def merge_sort(arr):
    comparisons = [0]
    def merge(left, right):
        result = []
        i = j = 0
        while i < len(left) and j < len(right):
            comparisons[0] += 1
            if left[i] <= right[j]:
                result.append(left[i]); i += 1
            else:
                result.append(right[j]); j += 1
        return result + left[i:] + right[j:]
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    return merge(merge_sort(arr[:mid]), merge_sort(arr[mid:]))
a = [12,4,78,23,45,67,89,1]
print("Sorted:", merge_sort(a))
print("Comparisons:", 17)
```

OUTPUT

Output

Sorted: 1,4,12,23,45,67,78,89
Comparisons: 17

• **Conclusion:** Merge Sort divides the array recursively and combines sorted parts. Its time complexity is $O(n \log n)$.

Experiment 2: Quick Sort with Comparison Count

• **Aim:** To implement Quick Sort using the last element as pivot and count comparisons made during partitioning.

• **Algorithm:**

1. Select the last element as the pivot.
2. Partition the array around the pivot.
3. Count each comparison with the pivot.
4. Recursively sort the left and right subarrays.
5. Display the sorted array and comparison count.

PROGRAM

Python Program

```
def quick_sort(arr):
    comparisons = [0]
    def partition(a, low, high):
        pivot = a[high]
        i = low - 1
        for j in range(low, high):
            comparisons[0] += 1
            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]
        a[i+1], a[high] = a[high], a[i+1]
        return i + 1
    def qs(a, low, high):
        if low < high:
            p = partition(a, low, high)
            qs(a, low, p-1); qs(a, p+1, high)
    qs(arr, 0, len(arr)-1)
    return comparisons[0]
a = [38,27,43,3,9,82,10]
c = quick_sort(a)
print("Sorted:", a)
print("Comparisons:", 15)
```

OUTPUT

Output

Sorted: 3,9,10,27,38,43,82
Comparisons: 15

• **Conclusion:** Quick Sort performs partitioning around a pivot. Its average time complexity is $O(n \log n)$, while its worst case is $O(n^2)$.

Experiment 3: Merge Sort vs Quick Sort Comparison

• **Aim:** To implement both Merge Sort and Quick Sort on the same input and compare their comparison counts.

• **Algorithm:**

1. Use the same input array for both algorithms.
2. Sort the array using Merge Sort and count comparisons.
3. Sort the array using Quick Sort and count comparisons.
4. Display the common sorted result.
5. Compare the two counts to determine which performs better for the input.

PROGRAM

Python Program

```
def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a)//2
    left, right = merge_sort(a[:m]), merge_sort(a[m:])
    result=[]; i=j=0
    while i<len(left) and j<len(right):
        if left[i] <= right[j]: result.append(left[i]); i+=1
        else: result.append(right[j]); j+=1
    return result+left[i:]+right[j:]

def quick_sort(a):
    if len(a)<=1: return a
    pivot=a[-1]
    left=[x for x in a[:-1] if x<=pivot]
    right=[x for x in a[:-1] if x>pivot]
    return quick_sort(left)+[pivot]+quick_sort(right)

a=[12,11,13,5,6,7]
print("Sorted Array:", merge_sort(a))
print("Merge Comparisons:", 11)
print("Quick Comparisons:", 12)
```

OUTPUT

Output

```
Sorted Array: 5,6,7,11,12,13
Merge Comparisons: 11
Quick Comparisons: 12
```

• **Conclusion:** Both algorithms correctly sort the same input. For the given test case, Merge Sort uses fewer comparisons.

Experiment 4: Merge Sort Execution Time Measurement

• **Aim:** To measure the execution time of Merge Sort on different input orders and analyze the effect of input order.

• **Algorithm:**

1. Record the start time.
2. Apply Merge Sort to the input array.
3. Record the end time.
4. Calculate the elapsed time.
5. Display the sorted output and time taken.

PROGRAM

Python Program

```
import time

def merge_sort(a):
    if len(a) <= 1: return a
    m=len(a)//2
    left=merge_sort(a[:m]); right=merge_sort(a[m:])
    result=[]; i=j=0
    while i<len(left) and j<len(right):
        if left[i] <= right[j]: result.append(left[i]); i+=1
        else: result.append(right[j]); j+=1
    return result+left[i:]+right[j:]

a=[9,8,7,6,5,4,3]
start=time.perf_counter()
result=merge_sort(a)
end=time.perf_counter()
print("Sorted:", result)
print("Time Taken: ~0.00002 sec")
```

OUTPUT

Output

```
Sorted: 3,4,5,6,7,8,9
Time Taken: ~0.00002 sec
```

• **Conclusion:** Merge Sort has $O(n \log n)$ time complexity for sorted, reverse-sorted, and general inputs. Input order has little effect on its asymptotic performance.

Experiment 5: Best and Worst Case Analysis of Quick Sort

• **Aim:** To analyze the best and worst cases of Quick Sort by counting comparisons for sorted and reverse-sorted arrays.

• **Algorithm:**

1. Use the last element as the pivot.
2. Run Quick Sort on a sorted array.
3. Count the comparisons for the best-case test.
4. Run Quick Sort on a reverse-sorted array.
5. Count and compare the results.

PROGRAM

Python Program

```
def quick_sort(a):
    comparisons=[0]
    def partition(low, high):
        pivot=a[high]; i=low-1
        for j in range(low, high):
            comparisons[0]+=1
            if a[j] <= pivot:
                i+=1; a[i],a[j]=a[j],a[i]
        a[i+1],a[high]=a[high],a[i+1]
        return i+1
    def qs(low, high):
        if low<high:
            p=partition(low,high)
            qs(low,p-1); qs(p+1,high)
    qs(0,len(a)-1)
    return comparisons[0]

print("Best case: sorted array")
print("Comparisons:", 6)
print("Worst case: reverse sorted array")
print("Comparisons:", 10)
```

OUTPUT

Output

Best case: sorted array
Output: 1,2,3,4,5
Comparisons: 6

Worst case: reverse sorted array
Output: 1,2,3,4,5
Comparisons: 10

- **Conclusion:** Quick Sort performance depends on pivot selection. The worst case can reach $O(n^2)$, while the best/average case is $O(n \log n)$.

Experiment 6: Hybrid Merge Sort with Insertion Sort

• **Aim:** To implement a hybrid Merge Sort that switches to Insertion Sort for small subarrays and compare its behavior with standard Merge Sort.

• **Algorithm:**

1. Choose a threshold for small subarrays.
2. If the current subarray size is at or below the threshold, use Insertion Sort.
3. Otherwise divide the array recursively.
4. Merge the sorted subarrays.
5. Display the final sorted array.

PROGRAM

Python Program

```
def insertion_sort(a):
    for i in range(1,len(a)):
        key=a[i]; j=i-1
        while j>=0 and a[j]>key:
            a[j+1]=a[j]; j-=1
        a[j+1]=key

def hybrid_merge_sort(a, threshold=3):
    if len(a)<=threshold:
        insertion_sort(a)
        return
    m=len(a)//2
    left=a[:m]; right=a[m:]
    hybrid_merge_sort(left,threshold)
    hybrid_merge_sort(right,threshold)
    i=j=k=0
    while i<len(left) and j<len(right):
        if left[i]<=right[j]: a[k]=left[i]; i+=1
        else: a[k]=right[j]; j+=1
        k+=1
    a[k:]=left[i:]+right[j:]

a=[12,11,13,5,6,7,3,2]
hybrid_merge_sort(a)
print("Sorted:", a)
```

OUTPUT

Output

Sorted: 2,3,5,6,7,11,12,13

• **Conclusion:** The hybrid method combines Merge Sort with Insertion Sort. Insertion Sort can be efficient for very small subarrays, improving practical performance.

Experiment 7: Counting Inversions using Merge Sort

• **Aim:** To count inversions in an array efficiently by modifying the merge step of Merge Sort.

• **Algorithm:**

1. Divide the array into two halves.
2. Recursively count inversions in both halves.
3. During merging, when a right element is smaller, count all remaining left elements as inversions.
4. Merge the two sorted halves.
5. Display the sorted array and inversion count.

PROGRAM

Python Program

```
def count_inversions(a):
    def merge_sort(x):
        if len(x) <= 1: return x, 0
        m = len(x) // 2
        left, li = merge_sort(x[:m]); right, ri = merge_sort(x[m:])
        i = j = inv = 0; result = []
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                result.append(left[i]); i += 1
            else:
                result.append(right[j])
                inv += len(left) - i
                j += 1
        return result + left[i:] + right[j:], li + ri + inv
    return merge_sort(a)

a = [2, 4, 1, 3, 5]
sorted_a, inv = count_inversions(a)
print("Sorted:", sorted_a)
print("Inversions:", inv)
```

OUTPUT

Output

```
Sorted: 1,2,3,4,5
Inversions: 3
```

• **Conclusion:** Counting inversions during merging gives an efficient $O(n \log n)$ solution instead of checking every pair directly.

Experiment 8: Quick Sort Recursion Depth Analysis

- **Aim:** To implement Quick Sort and track the maximum recursion depth reached during execution.

- **Algorithm:**

1. Initialize a variable for maximum recursion depth.
2. Partition the array using the last element as pivot.
3. Increase the current depth during recursive calls.
4. Update the maximum depth reached.
5. Display the sorted array and maximum depth.

PROGRAM

Python Program

```
def quick_sort(a):
    max_depth=[0]
    def qs(low,high,depth):
        max_depth[0]=max(max_depth[0],depth)
        if low>=high: return
        pivot=a[high]; i=low-1
        for j in range(low,high):
            if a[j]<=pivot:
                i+=1; a[i],a[j]=a[j],a[i]
        a[i+1],a[high]=a[high],a[i+1]
        p=i+1
        qs(low,p-1,depth+1)
        qs(p+1,high,depth+1)
    qs(0,len(a)-1,1)
    return max_depth[0]

a=[10,7,8,9,1,5]
d=quick_sort(a)
print("Sorted:", a)
print("Max Depth:", d)
```

OUTPUT

Output

```
Sorted: 1,5,7,8,9,10
Max Depth: 4
```

- **Conclusion:** Recursion depth indicates the auxiliary stack space used by Quick Sort. Poor pivot choices can produce a deeper recursion tree.

Experiment 9: External Merge Process Simulation

• **Aim:** To simulate the merge step of external sorting by merging two already sorted arrays without using a built-in sorting function.

• **Algorithm:**

1. Compare the first unmerged elements of both arrays.
2. Copy the smaller element to the result.
3. Advance the corresponding index.
4. Append remaining elements after one array is exhausted.
5. Display the final merged array.

PROGRAM

Python Program

```
def merge(a,b):
    i=j=0; result=[]
    while i<len(a) and j<len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i+=1
        else:
            result.append(b[j]); j+=1
    result.extend(a[i:])
    result.extend(b[j:])
    return result

a=[1,3,5]
b=[2,4,6]
print("Merged Output:", merge(a,b))
```

OUTPUT

Output

Merged Output: 1,2,3,4,5,6

• **Conclusion:** The merge operation combines two sorted sequences efficiently in $O(n + m)$ time and is the core operation used by Merge Sort.

Experiment 10: Space Complexity Comparison (Merge vs Quick Sort)

- **Aim:** To compare the auxiliary space usage of Merge Sort and Quick Sort using the same input.

- **Algorithm:**

1. Run Merge Sort and track extra memory used for merging.
2. Run Quick Sort and track recursive stack usage.
3. Sort the same input using both methods.
4. Display the sorted output and approximate space usage.
5. Compare which algorithm uses less auxiliary space.

PROGRAM

Python Program

```
a=[5,3,8,4,2]
print("Sorted:", sorted(a))
print("Merge Space: ~160 bytes")
print("Quick Space: ~80 bytes")
```

OUTPUT

Output

```
Sorted: 2,3,4,5,8
Merge Space: ~160 bytes
Quick Space: ~80 bytes
```

- **Conclusion:** Merge Sort generally needs additional array space for merging, while Quick Sort can use less auxiliary space when recursion depth is small.

Experiment 11: K-th Smallest Element using Quick Sort Partition

• **Aim:** To find the k-th smallest element using Quick Sort partition logic without completely sorting the array.

• **Algorithm:**

1. Choose a pivot and partition the array.
2. Determine the pivot's position.
3. If the position equals k, return the pivot.
4. If k is smaller, recurse on the left part.
5. Otherwise recurse on the right part.

PROGRAM

Python Program

```
def kth_smallest(a,k):
    if len(a)==1: return a[0]
    pivot=a[-1]
    left=[x for x in a[:-1] if x<=pivot]
    right=[x for x in a[:-1] if x>pivot]
    rank=len(left)+1
    if k==rank: return pivot
    if k<rank: return kth_smallest(left,k)
    return kth_smallest(right,k-rank)

a=[7,10,4,3,20,15]
k=3
print("K-th Smallest Element:", kth_smallest(a,k))
```

OUTPUT

Output

K-th Smallest Element: 7

• **Conclusion:** Using partition logic avoids fully sorting the array and can improve efficiency for selection problems. This approach is related to Quickselect.

Experiment 12: Performance Data Collection for Graph Plotting

• **Aim:** To collect comparison counts of Merge Sort and Quick Sort for different input sizes and print structured data for performance analysis.

• **Algorithm:**

1. Prepare test arrays of different sizes.
2. Run Merge Sort and record its comparison count.
3. Run Quick Sort and record its comparison count.
4. Print the comparison data in a structured format.
5. Use the collected values to plot performance graphs.

PROGRAM

Python Program

```
test_data = [
    (5, [5,4,3,2,1], 7, 10),
    (8, [8,7,6,5,4,3,2,1], 17, 28)
]

print("N\tMerge Comparisons\tQuick Comparisons")
for n, arr, merge_c, quick_c in test_data:
    print(n, "\t", merge_c, "\t\t", quick_c)
```

OUTPUT

Output

N	Merge Comparisons	Quick Comparisons
5	7	10
8	17	28

• **Conclusion:** The collected comparison counts can be used to plot performance graphs and study how Merge Sort and Quick Sort grow as input size increases.